

Overview

🎯 In this course, you will learn about advanced hooking in Squish by familiarizing yourself with the concept of application context, learning how to test multiple AUTs and attach to running AUTs.

[GitHub - qt-learning/Advanced-Hooking · GitHub](#)

[Squish API | Squish 9.2](#)

[How to Test Multiple AUTs from a Single Test Script, Using ApplicationContext | Squish 9.2](#)

[Testing Multiple Applications Using Separate Squish Packages \(In One Test Case\)](#)

[Attaching to Running Applications | Squish 9.2](#)

What Will I Learn

1. What is Application Context, and how to use it to query information.
 2. How to test several applications in the same test script.
 3. How to test applications that have been built with different toolkits in the same test script.
 4. How to attach and test an application started outside Squish.
-

Application Context

In general, an AUT is started in each test case with a call to the function `startApplication()`, with the name of the AUT to start passed as argument. In this function, it is also possible to use the absolute path to an executable or command line options for the AUT.

The function returns a handle to the AUT's application context. The **ApplicationContext** object is created whenever an AUT is started. This object allows Squish to query the AUT for the command line it was launched with, the used memory, what the AUT writes to stdout and stderr among other useful information.

None

```
def init():

    appContext = startApplication("addressbook fr")

    test.log("Application started with command \"%s\" at time %f in directory %s" %
            (appContext.commandLine, appContext.startTime, appContext.cwd))
```

The following functions and read-only properties are available from the application context:

commandLine	The command line that the application was started with.
cwd	The application's current working directory
environmentVariable(name)	Returns the value of the AUT process environment variable called name or an empty string if no such environment variable exists.
host	Computer/ host name or TCP/IP address of the computer where the squishserver process is running and which this application context is connected to.
isFrozen(timeout)	Returns a false value if the application responds before the timeout (in seconds) expired.
name	The application's name.
pid	The application's process identifier
port	The TCP port number of the squishserver process which the application context is connected to.
readStderr()	Returns everything which the application has written to the stderr stream since the last call to readStderr, or since the application started.
readStdout()	Returns everything which the application has written to the stdout stream since the last call to readStdout, or since the application started.
startTime	The time when the application was started.
totalTime	The amount of time that the AUT process has been scheduled in user and kernel modes measured in milliseconds.
usedMemory	The amount of memory used by the AUT process. For Windows operating systems, the value corresponds to the working set size in bytes. For Linux operating systems, the value corresponds to the resident set size (RSS), in bytes. For macOS, it is also the RSS, but in Kbytes.
osName	The name of the OS the AUT is running on.

The ApplicationContext can also be accessed by calling the currentApplicationContext()(opens in a new tab) function. This function returns the application context of the currently active AUT.

Python

```
ctx = currentApplicationContext()

    test.log("%s is current application context with id %d on port  
%d and host %s" %(ctx.name, ctx.pid, ctx.port, ctx.host))

    test.log("Memory: %d" % ctx.usedMemory)
```

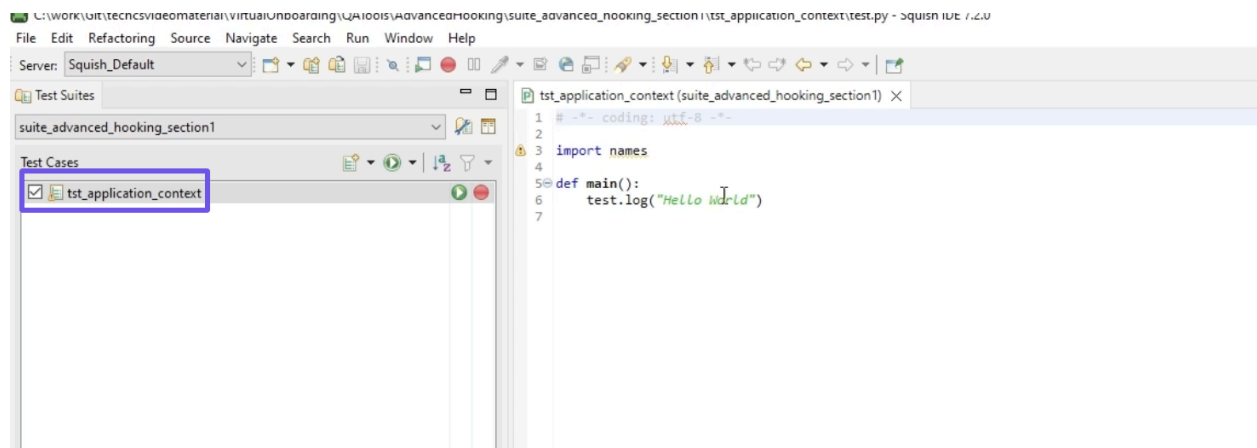
After an AUT has been started with `startApplication()`(opens in a new tab), the function `detach()`(opens in a new tab) can be used to tell Squish to close it. The function can be used when an application needs to be closed during the test execution. After detaching from an AUT, its objects will no longer be accessible.

Python

```
def cleanup():
    ctx = currentApplicationContext()
    test.log("STDOUT", ctx.readStdout())
    test.warning("STDERR", ctx.readStderr())
    ctx.detach()
```

Steps : How to Use the Application Context Object to Query Information About the AUT

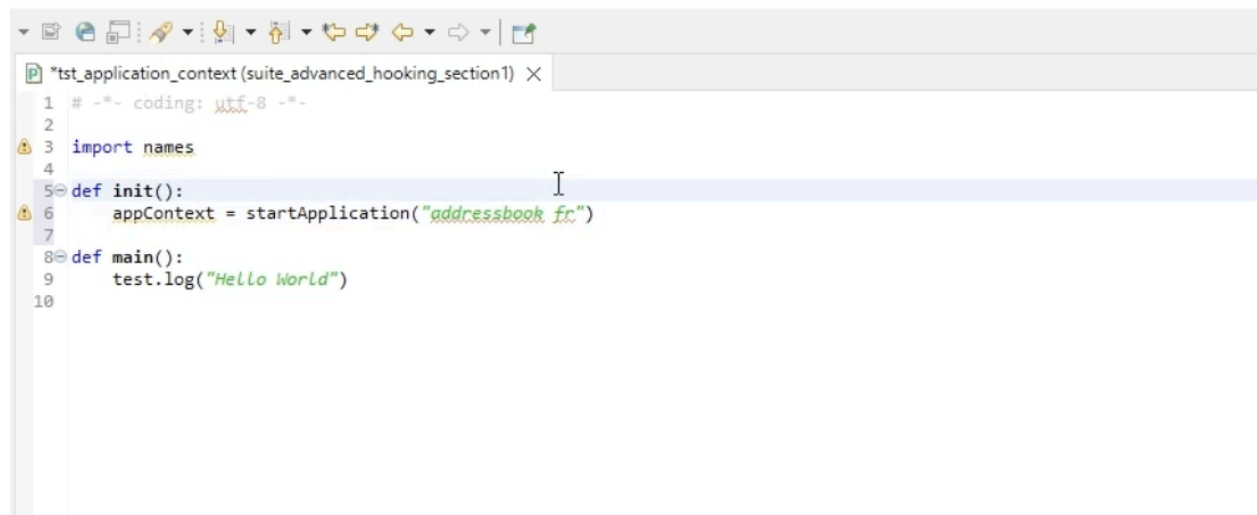
STEP 01 = Creating a new test case



We started by creating a new test case named `tst_application_context` in our test suite. We did not record anything but directly wrote the test script in the Python code editor.

STEP 02 = Defining the `init()` function

suite_advanced_hooking_section1\tst_application_context\tst.py - Squish IDE 7.2.0



We first defined the `init()` function. Like `main()`, `init()` is a test-created special function that serves a specific purpose in Squish. If the function is defined in the script, it will be called before `main()`, and therefore, it is used when we want to do some initialization before executing the test.

In this function, we started our Address Book application and then used the application context object returned to get information about the AUT we just started. Instead of simply calling `startApplication()`, we saved the return value to a variable called `appContext`.

```
appContext = startApplication("addressbook fr")
```

Using the application context, we could obtain information about how the application was started. In the `startApplication` function, we added the option "fr" to start the application in French.

STEP 03 = Adding a log call to the init() function

```
*tst_application_context(suite_advanced_hooking_section1) X
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def init():
6     appContext = startApplication("addressbook_fc")
7     test.log("Application started with command \"%s\" at time %f in directory %s" % (appContext.commandLine, appContext.startTime, appContext.cwd))
8
9 def main():
10     test.log("Hello World")
11
```

We checked that the option was correctly used by querying the command line which was used to start the AUT.

We also checked when the AUT was started and its current working directory. All this information was obtained using the application context, and we printed it in the log using the test.log function.

Then we put the command line, start time, and current working directory of appContext as parameters for our log string.

```
test.log("Application started with command \"%s\" at time %f in directory %s" %
(appContext.commandLine, appContext.startTime, appContext.cwd))
```

STEP 04 = Defining a cleanup() function

```
*tst_application_context(suite_advanced_hooking_section1) X
1 # -*- coding: utf-8 -*-
2
3 import names
4
5 def init():
6     appContext = startApplication("addressbook_fc")
7     test.log("Application started with command \"%s\" at time %f in directory %s" % (appContext.commandLine, appContext.startTime, appContext.cwd))
8
9 def cleanup():
10     ctx = currentApplicationContext()
11     test.log("STDOUT", ctx.readStdout())
12     test.warning("STDERR", ctx.readStderr())
13     ctx.detach()
14
15 def main():
16     test.log("Hello World")
17
```

Next, we defined a cleanup function that would be called at the end of our test. cleanup() is a special function which will be automatically called after the main() function is completed, if defined.

We added def cleanup(). In this function, we wanted to query and print standard output and standard error message of the current application context before closing the AUT.

First, we got the application context by writing:

```
ctx = currentApplicationContext()
```

Then, we printed the standard output and the standard error.

```
test.log("STDOUT", ctx.readStdout())
```

```
test.warning("STDERR", ctx.readStderr())
```

Lastly, we wanted to make sure we closed the current AUT by detaching from it with the detach function.

```
ctx.detach()
```

STEP 05 = Writing the main() function

```
1 > import names
2
3
4
5 def init():
6     appContext = startApplication("addressbook.fc")
7     test.log("Application started with command '%s' at time %f in directory %s" % (appContext.commandLine, appContext.startTime, appContext.cwd))
8
9 def cleanup():
10    ctx = currentApplicationContext()
11    test.log("STDOUT", ctx.readStdout())
12    test.warning("STDERR", ctx.readStderr())
13    ctx.detach()
14
15 def main():
16    ctx = currentApplicationContext()
17    test.log("%s is current application context with id %d on port %d and host %s" % (ctx.name, ctx.pid, ctx.port, ctx.host))
18    test.log("Memory used %d" % ctx.usedMemory)
19
20
```

Next, we wrote our main function. We removed the initial call to test log present in test script template.

In this function, we still used the application context to get information about the AUT that had started in the "init" function. First, we got the current application context:

```
ctx = currentApplicationContext()
```

Then, we logged the name of the AUT, its processor ID, the port and host of the squish server, and the AUT memory:

```
test.log("%s is current application context with id %d on port %d and host %s" % (ctx.name, ctx.pid, ctx.port, ctx.host))
```

```
test.log("Memory: %d" % ctx.usedMemory)
```

STEP 06 = Running the test

Recent Test Results	
Result	Message
▼ TestSuite	suite_advanced_hooking_section1
▼ TestCase	tst_application_context
Log	Application started with command "C:\Squish\Squish_for_Qt65x_7_2_0\examples\qt\addressbook\addressbook.exe fr" at time 1706793000.000000 in deirectory C:\Squish\Squish_for_Qt65x_7_...
Log	addressbook is current application context with id 19096 on port 61643 and host 127.0.0.1
Log	Memory used 56008704
Log	STDOUT
> Warning	STDERR

Then, we ran our test.

When the test finished, we saw that some information was logged to the test results. Firstly, we had the information regarding the start of the application. This was called in the init function. As we saw, the AUT was started by a call to the executable and with the fr option. The application was started in the executable directory.

Then, we called our main function that queried some information about the running application. The name was address book, and the host was the local host, as expected. The memory was also logged.

Finally, thanks to the call of "clean up" we got the standard error. We saw everything the application had written to the standard error stream since the application started.

Testing Multiple AUTs

The `startApplication()`(opens in a new tab) function can be called several times in the same test script, so it is possible to start several instances of the same application or start multiple different applications with one script. Being able to start several AUTs that can all be evaluated in the same test script is essential for testing client/server systems, for instance.

The most recently started application is considered the active application: its application context will be returned by `currentApplicationContext()`(opens in a new tab) and all the commands in the script will be applied to this AUT. Other application contexts can be obtained by using the `applicationContextList()`(opens in a new tab) function, which returns the context objects of all AUTs started or attached to in the script.

The `setApplicationContext()`(opens in a new tab) function can be used to set an inactive AUT as active. When testing with multiple AUTs in a same test case, it is best to start all AUTs at the beginning of the test, then change the active application based on which AUT should be interacted with.

In the following snippet, two instances of the same AUT are started, and then `setApplicationContext()` is called to let Squish interact with the first instance:

None

```
app1 = startApplication("addressbook")

app2 = startApplication("addressbook")

apps = applicationContextList()

for app in apps:

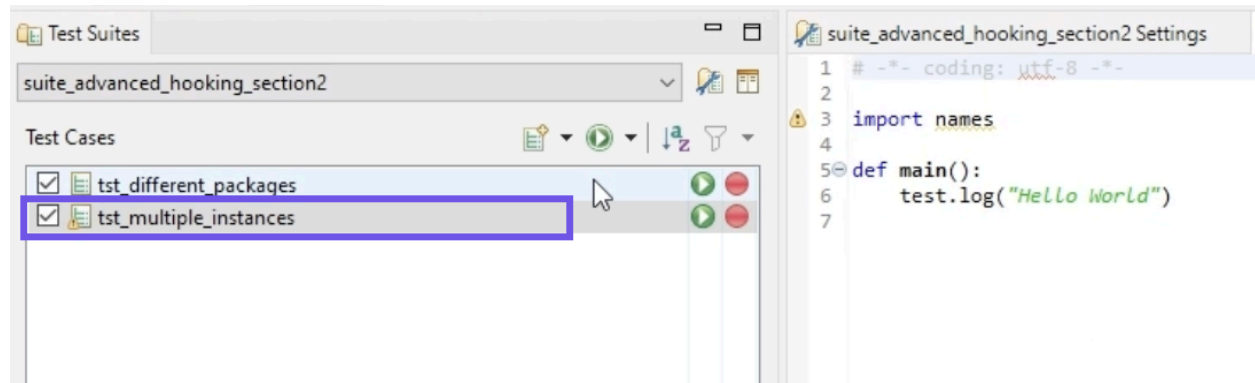
    test.log("%s is running with id %d on port %d with host %s" %
(app.name,
    app.pid, app.port, app.host))

setApplicationContext(app1)

clickButton(waitForObject(names.address_Book_New_QToolButton))
```

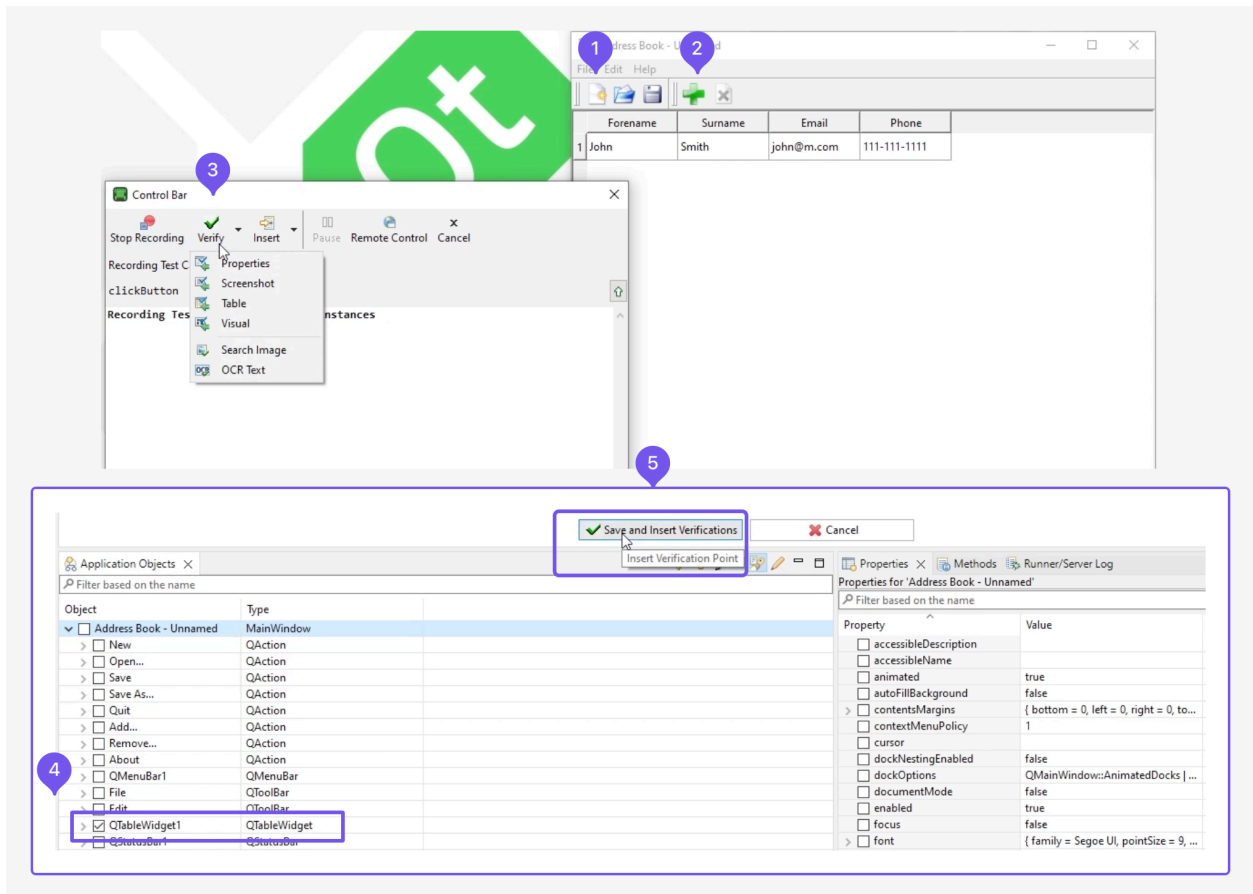
Steps - Test Several Applications in the Same Test Script

STEP 01 = Creating a test case



We started by creating a new test case named `tst_multiple_instances` in our test suite. In this test, we started two instances of the AUT and interacted with both.

STEP 02 = Record a test



We started by recording our test for one instance of the address book application. The purpose was simply to set up the format for our test which would run on multiple instances of the addressbook.

We clicked on the record button, and started the AUT.

1. Then we created a new address book.

2. We added a new entry to it.

3-5. Then, we perform a table verification to confirm the entry was correctly added.

Lastly, we stopped the recording.

STEP 03 = Modifying the test script

Next, we modified the test script to test two instances of the AUT. Starting several instances of the application in a test script is straightforward; we just needed to add several calls to `startApplication()`.

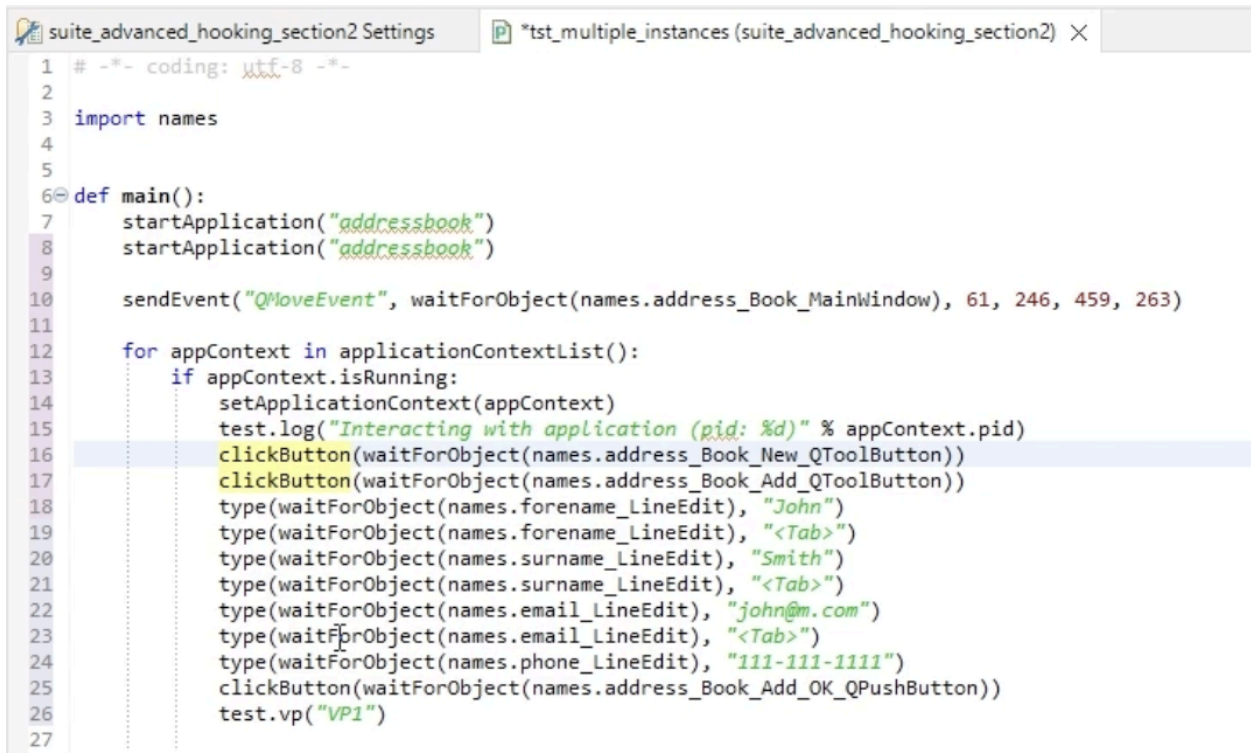
So, we first copy-pasted the first line of the main function.

```
startApplication("addressbook")
```

The two applications would be opened in the same place, so we moved the second one using a `QMoveEvent`. We used `sendEvent` to trigger a move event, which is the first argument of `sendEvent`. The second argument of the function is the object we wanted to send the event to, here, the main window. The remaining arguments were the old and new position of the object to move.

```
sendEvent("QMoveEvent", waitForObject(names.address_Book_MainWindow), 61, 246, 459, 263)
```

STEP 04 = Adding a **for** loop

The image shows a screenshot of a code editor with a file named `*tst_multiple_instances (suite_advanced_hooking_section2)`. The code is a Python test function `main()` that starts two instances of an application named "addressbook". It then sends a `QMoveEvent` to the main window of the second instance. Finally, it enters a `for` loop that iterates over `applicationContextList()`. Inside the loop, it checks if the application context is running, sets the application context, and then performs a series of actions: clicking the "New" button, clicking the "Add" button, typing "John" into the forename field, pressing Tab, typing "Smith" into the surname field, pressing Tab, typing "john@m.com" into the email field, pressing Tab, typing "111-111-1111" into the phone field, and finally clicking the "Add OK" button. The test ends with `test.vp("VP1")`.

```
1 # -*- coding: utf-8 -*-
2
3 import names
4
5
6 def main():
7     startApplication("addressbook")
8     startApplication("addressbook")
9
10    sendEvent("QMoveEvent", waitForObject(names.address_Book_MainWindow), 61, 246, 459, 263)
11
12    for appContext in applicationContextList():
13        if appContext.isRunning:
14            setApplicationContext(appContext)
15            test.log("Interacting with application (pid: %d)" % appContext.pid)
16            clickButton(waitForObject(names.address_Book_New_QToolButton))
17            clickButton(waitForObject(names.address_Book_Add_QToolButton))
18            type(waitForObject(names.forename_LineEdit), "John")
19            type(waitForObject(names.forename_LineEdit), "<Tab>")
20            type(waitForObject(names.surname_LineEdit), "Smith")
21            type(waitForObject(names.surname_LineEdit), "<Tab>")
22            type(waitForObject(names.email_LineEdit), "john@m.com")
23            type(waitForObject(names.email_LineEdit), "<Tab>")
24            type(waitForObject(names.phone_LineEdit), "111-111-1111")
25            clickButton(waitForObject(names.address_Book_Add_OK_QPushButton))
26            test.vp("VP1")
27
```

Next, we created a “for” loop to perform the same test but on the two instances. To do that, we looped through the list of application context objects. Each time we called `startApplication()`, we added an object in `applicationContextList`:

```
for appContext in applicationContextList():
```

Then we checked that the application was running with:

```
if appContext.isRunning:
```

Then since by default Squish will interact with the last application started by `startApplication()`, we needed to change the current application context.

We did that by calling `setApplicationContext()`

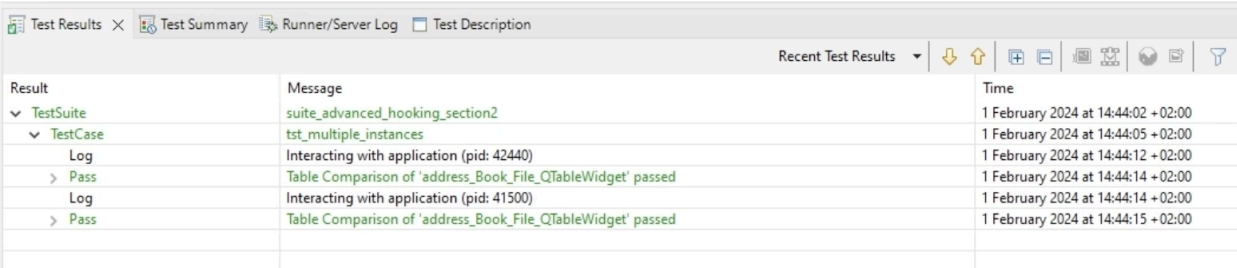
```
setApplicationContext(appContext)
```

Since we had the application context in a variable, we used it to get information about the AUT. We made sure we interacted with the two AUTs by just printing the process id to the log :

```
test.log("Interacting with application (pid: %d)" % appContext.pid)
```

The last thing we needed to do was to put the recorded code into the “for” loop.

STEP 05 = Running the test



Recent Test Results		
Result	Message	Time
TestSuite	suite_advanced_hooking_section2	1 February 2024 at 14:44:02 +02:00
TestSuite	tst_multiple_instances	1 February 2024 at 14:44:05 +02:00
Log	Interacting with application (pid: 42440)	1 February 2024 at 14:44:12 +02:00
Pass	Table Comparison of 'address_Book_File_QTableWidget' passed	1 February 2024 at 14:44:14 +02:00
Log	Interacting with application (pid: 41500)	1 February 2024 at 14:44:14 +02:00
Pass	Table Comparison of 'address_Book_File_QTableWidget' passed	1 February 2024 at 14:44:15 +02:00

We ran the test again. We saw the two instances were started, and we tested them both.

Testing two applications that have been built with different toolkits in the same test script

With the package Squish for Qt, it is also possible to start AUTs that are built with different GUI toolkits (Tk or iOS) from the same test script. In that case the wrappers corresponding to the toolkit the AUT was built with must be set before starting the AUT. This is done using the `setWrappersForApplication()`(opens in a new tab) function.

Furthermore, when testing AUTs built with different toolkits that are not supported by the same Squish packages (for instance testing a Qt and a Java application in the test script), the required Squish packages must all be installed, and the respective Squish tools must be used. Then, multiple Squish servers must be configured and started before the execution of the script. The

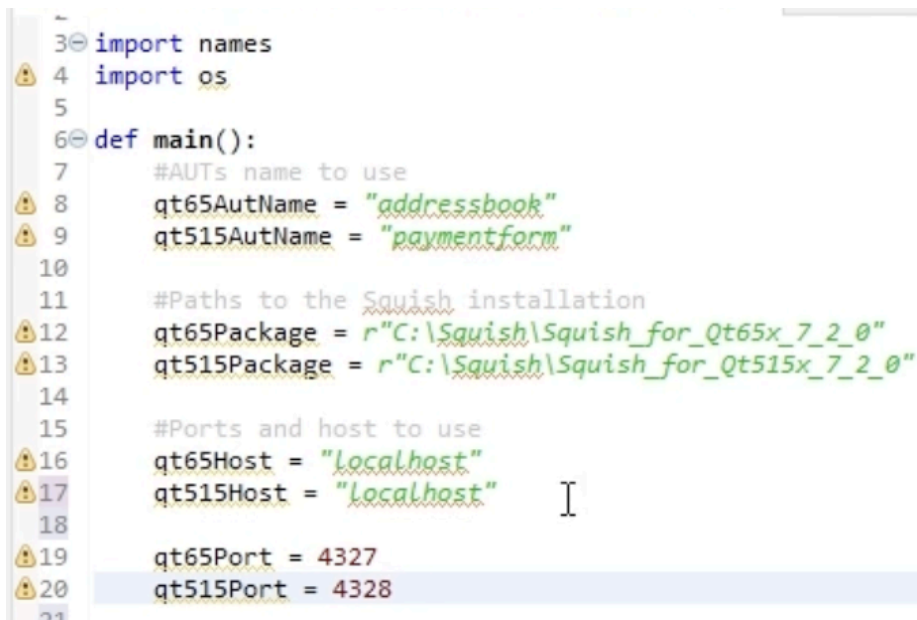
method is shown in the video below, but for more details about testing multiple AUTs using separate Squish Packages, see the following blog post:

[Testing Multiple Applications Using Separate Squish Packages \(In One Test Case\).](#)

Steps - Testing Two Applications that Have been Built with Different Toolkits in the Same Test Script

STEP 00 =

STEP 01-



```
3 import names
4 import os
5
6 def main():
7     #AUTs name to use
8     qt65AutName = "addressbook"
9     qt515AutName = "paymentform"
10
11     #Paths to the Squish installation
12     qt65Package = r"C:\Squish\Squish_for_Qt65x_7_2_0"
13     qt515Package = r"C:\Squish\Squish_for_Qt515x_7_2_0"
14
15     #Ports and host to use
16     qt65Host = "localhost"
17     qt515Host = "localhost"
18
19     qt65Port = 4327
20     qt515Port = 4328
21
```

We had already created a test case to test the two applications. We started with `tst_different_packages`.

First, we set the name of the AUTs we wanted to test and the path to the Squish packages. So, the address book in Qt 6.5 and the payment form in Qt 5.15.

```
qt65AutName = "addressbook"
```

```
qt515AutName = "paymentform"
```

```
qt65Package = r"C:\Squish\Squish_for_Qt65x_7_2_0"
```

```
qt515Package = r"C:\Squish\Squish_for_Qt515x_7_2_0"
```

Then, we set the ports and host used when the two squishservers were started. Since the packages were different, we needed to start two squishserver in this test. We started them on the same local host but with two different ports:

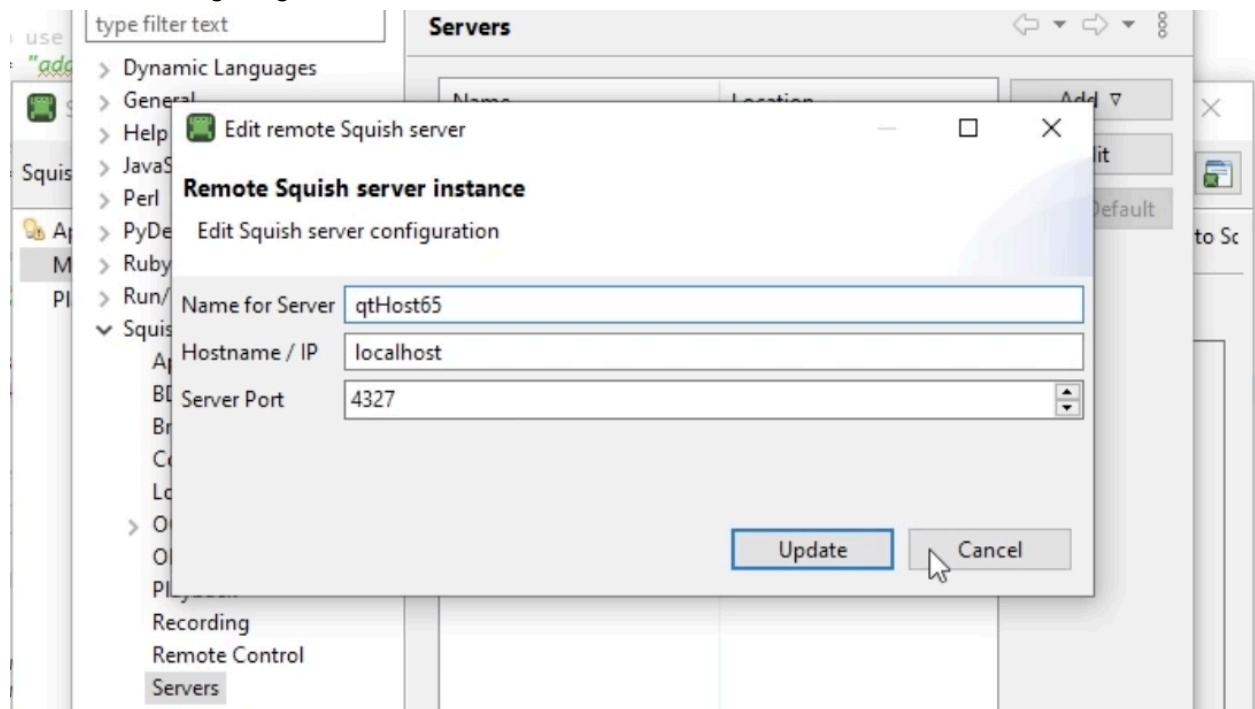
```
qt65Host = "localhost"
```

```
qt515Host = "localhost"
```

```
qt65Port = 4327
```

```
qt515Port = 4328
```

STEP 02- Configuring the servers

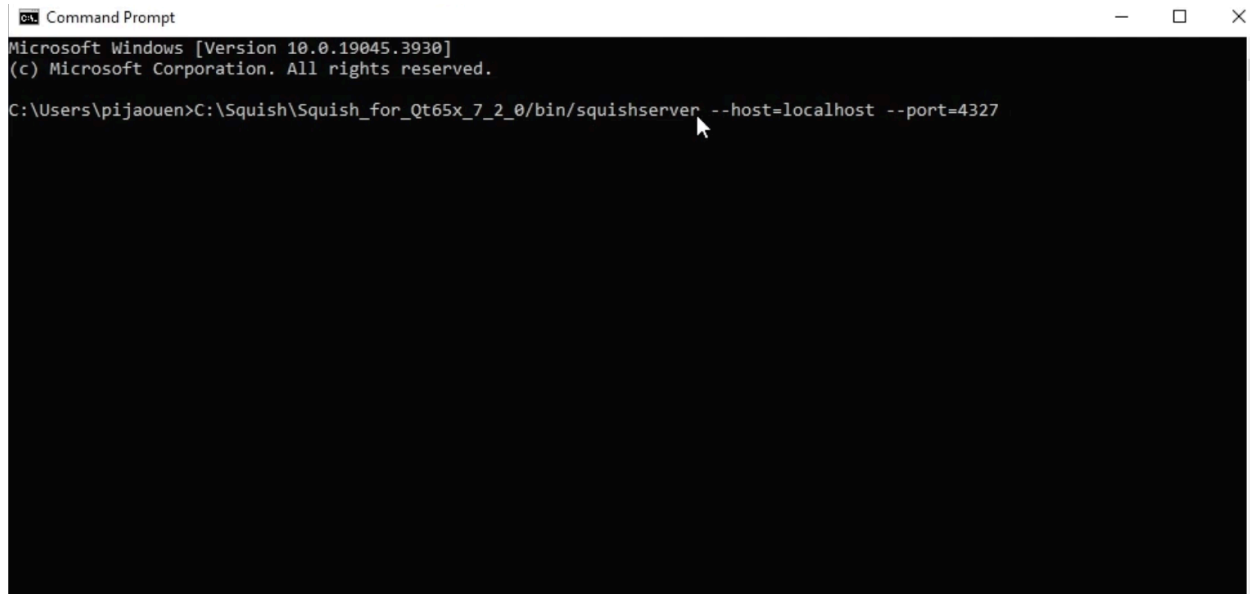


Next, we configured the servers. To do this, we went to Edit, Preferences, Servers.

Here, we needed to have two remote servers. Both would be on the local host but with two ports. One for the Squish for Qt 6.5 server and one for the Squish for Qt 5.15 server. Once this was done, we saved the configuration.

Then, we moved back to the Squish IDE. There, we selected the first remote server we had added. This was done from the 'Server' drop-down menu, located in the upper left-hand corner of the IDE.

STEP 03- Starting the Servers



```
Command Prompt
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pijaouen>C:\Squish\Squish_for_Qt65x_7_2_0\bin\squishserver --host=localhost --port=4327
```

The next step was to start the two servers we had configured. It is important to execute the servers independently of the script and of squishrunner. If we had started up squishserver within the test case, squishserver execution would have depended on squishrunner. This was not the intended method of execution.

So, we opened two command prompts. We started the squish server for Squish for Qt 6.5 with the host and port we had set in the Server edition wizard.

```
#SQUISHDIR/bin/squishserver --port=qt65Port --host=qt65Host
```

We did the same for Squish for Qt 5.15 in the second console prompt.

```
#SQUISHDIR/bin/squishserver --port=qt515Port --host=qt515Host
```

STEP 04- Starting and testing the Qt 6.5 app

```

22
23 #Starting the servers via the command line
24 #SQUISHDIR/bin/squishserver --port=qt65Port --host=qt65Host
25 #SQUISHDIR/bin/squishserver --port=qt515Port --host=qt515Host
26
27 #Starting and testing the Qt 6.5 app and testing it
28 os.environ["SQUISH_PREFIX"] = qt65Package
29 testSettings.setWrappersForApplication(qt65AutName, ["Qt"])
30 qt65Context = startApplication(qt65AutName, qt65Host, qt65Port)
31
32
33 clickButton(waitForObject(names.address_Book_New_QToolButton))
34 clickButton(waitForObject(names.address_Book_Add_QToolButton))
35 type(waitForObject(names.forename_LineEdit), "John")
36 type(waitForObject(names.forename_LineEdit), "<Tab>")
37 type(waitForObject(names.surname_LineEdit), "Smith")
38 type(waitForObject(names.surname_LineEdit), "<Tab>")
39 type(waitForObject(names.email_LineEdit), "john@m.com")
40 type(waitForObject(names.email_LineEdit), "<Tab>")
41 type(waitForObject(names.phone_LineEdit), "123123")
42 clickButton(waitForObject(names.address_Book_Add_OK_QPushButton))
43 test.vp("VP1")
44

```

When our servers were started, we went back to the Squish IDE. We went over the steps for testing the two applications.

First, we tested the application developed with Qt 6.5. We needed to change the environment variable SQUISH_PREFIX to be able to execute test script commands from the Squish for Qt 6.5 package.

```
os.environ["SQUISH_PREFIX"] = qt65Package
```

Then, we set the wrapper for our application. In this case the default wrapper is "Qt", so we did not need to set the wrapper explicitly. However, in any case where the desired wrapper is different from the default, this line would be required, so we called the function for demonstration purposes.

```
testSettings.setWrappersForApplication(qt65AutName, ["Qt"])
```

Finally, we started the AUT using the port and host corresponding to the correct server.

```
qt65Context = startApplication(qt65AutName, qt65Host, qt65Port)
```

In the following lines, we could see we had some code to add an entry to our address book application. The purpose was just to show we were successfully connected to the Qt 6.5 AUT and could interact with it. Then we did a table verification.

STEP 05- Starting and testing the Qt 5.15 app

```

44
45 #Starting and testing the Qt 5.15 app and testing it
46 os.environ["Squish_PREFIX"] = qt515Package
47 testSettings.setWrappersForApplication(qt515AutName, ["Qt"])
48 qt515Context = startApplication(qt515AutName, qt515Host, qt515Port)
49
50
51 spinUp(waitForObject(names.this_Payment_QSpinBox))
52 spinUp(waitForObject(names.this_Payment_QSpinBox))
53 doubleClick(waitForObject(names.this_Payment_QSpinBox), 226, 7, Qt.NoModifier, Qt.LeftButton)
54 spinUp(waitForObject(names.this_Payment_QSpinBox))
55 doubleClick(waitForObject(names.this_Payment_QSpinBox), 226, 7, Qt.NoModifier, Qt.LeftButton)
56 test.compare(str(waitForObjectExists(names.make_Payment_qt_spinbox_lineedit_QLineEdit).text), "$ 107")
57
58 #Go back to Qt 6.5 app
59
60 sendEvent("QMoveEvent", waitForObject(names.address_Book_MainWindow), 61, 246, 459, 263)
61
62 #Closing the servers

```

Then, we repeated the same process to start the Qt 5.15 application, execute the commands from the Squish for Qt 5.15 edition and connect to the appropriate server.

```
os.environ["SQUISH_PREFIX"] = qt515Package
```

```
testSettings.setWrappersForApplication(qt515AutName, ["Qt"])
```

```
qt515Context = startApplication(qt515AutName, qt515Host, qt515Port)
```

The next lines of our code executed some testing on the Qt 5.15 application. We simply interacted with the spin box on the payment form to increase the value displayed and tested that this value was what we expected. This test would pass as long as we had successfully connected to the Qt 5.15 application.

STEP 06- Switching back to the Qt 6.5 app

```

47 testSettings.setWrappersForApplication(qt515AutName, ["Qt"])
48 qt515Context = startApplication(qt515AutName, qt515Host, qt515Port)
49
50
51 spinUp(waitForObject(names.this_Payment_QSpinBox))
52 spinUp(waitForObject(names.this_Payment_QSpinBox))
53 doubleClick(waitForObject(names.this_Payment_QSpinBox), 226, 7, Qt.NoModifier, Qt.LeftButton)
54 spinUp(waitForObject(names.this_Payment_QSpinBox))
55 doubleClick(waitForObject(names.this_Payment_QSpinBox), 226, 7, Qt.NoModifier, Qt.LeftButton)
56 test.compare(str(waitForObjectExists(names.make_Payment_qt_spinbox_lineedit_QLineEdit).text), "$ 107")
57
58 #Go back to Qt 6.5 app
59 setApplicationContext(qt65Context)
60 sendEvent("QMoveEvent", waitForObject(names.address_Book_MainWindow), 61, 246, 459, 263)
61
62 #Closing the servers

```

Lastly, we switched back to interacting with the Qt 6.5 app with the `setApplicationContext()` function.

```
setApplicationContext(qt65Context)
```

STEP 07- Running the test

Lastly, we switched back to interacting with the Qt 6.5 app with the **setApplicationContext()** function.

setApplicationContext(qt65Context)

Result	Message
▼ TestSuite	suite_advanced_hooking_section2
▼ TestCase	tst_different_packages
▼ Pass	Table Comparison of 'address_Book_File_QTableWidget' passed
Detail	Table contents appear to be equal.
▼ Pass	Comparison
Detail	'\$ 107' and '\$ 107' are equal

We ran our test to confirm it worked. We saw that we started the two applications and successfully interacted with them. Once our test was done, we closed our servers.

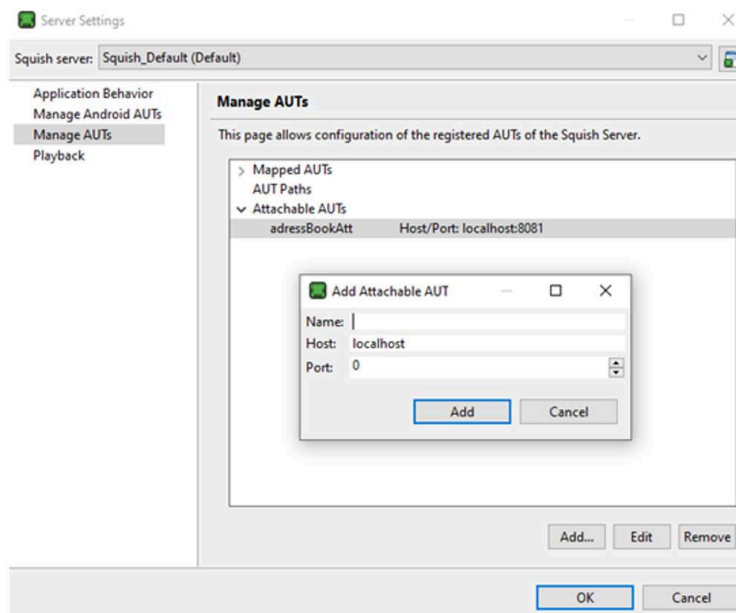
Attaching to a Running AUT

AUTs started with [startApplication\(\)](#) are closed at the end of a test case. When the AUT is started outside of the test case, it will remain open after the test is complete. Unlike when using [startApplication\(\)](#) however, Squish can also attach to an application opened outside of the test case. This is done with the function [attachToApplication\(\)](#). The AUT must be started with the [startaut](#) command line tool to be attachable. It can be started in this way:

```
SQUISHDIR\bin>startaut --port={port} {path to aut}
```

The AUT must be registered as an attachable AUT to call the [attachToApplication\(\)](#) function successfully.

Add Attachable AUT Wizard



Attachable AUTs can be registered in: Edit > Server Settings... > Attachable AUTs > Add... The Name (name of the AUT) will be the argument passed in

attachToApplication(). The Port must match the port number used when starting the AUT with startaut. The Host is the address of the machine where the attachable AUT is running.

Once an AUT is registered as an attachable application, Squish can attach to it using the **attachToApplication()** function. Like for **startApplication()**, this function returns an **ApplicationContext** so the same principles explained in the Sections Application Context and Testing Multiple AUTs apply.

```
None
def main():

    #Application must be started from a command-line tool:

    #SQUISHDIR\bin>startaut --port={port} {path to aut}

    app = attachToApplication("att_addressBook")
```

When working with an attached application, the **detach()** function will detach from the application but leave it running. The application can then be attached to with **attachToApplication()** again.

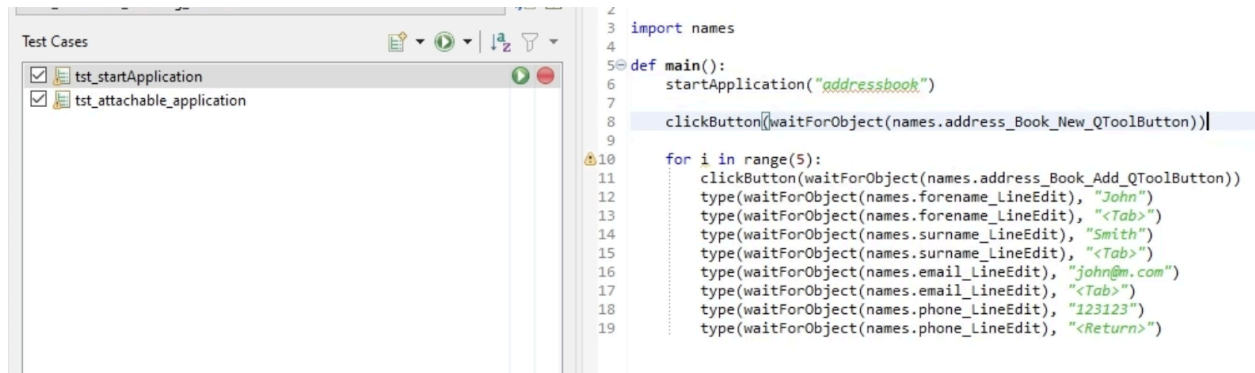
Steps

STEP 0- Starting point

In the video, we demonstrated how to attach and test an application started outside of Squish. This allows for testing an AUT on a machine other than the test machine, as the AUT can be connected to the test over a network connection.

We used the Address Book example. The application can be found in the examples subdirectory in the Squish installation

STEP 1- Creating a test case and exploring the already created test

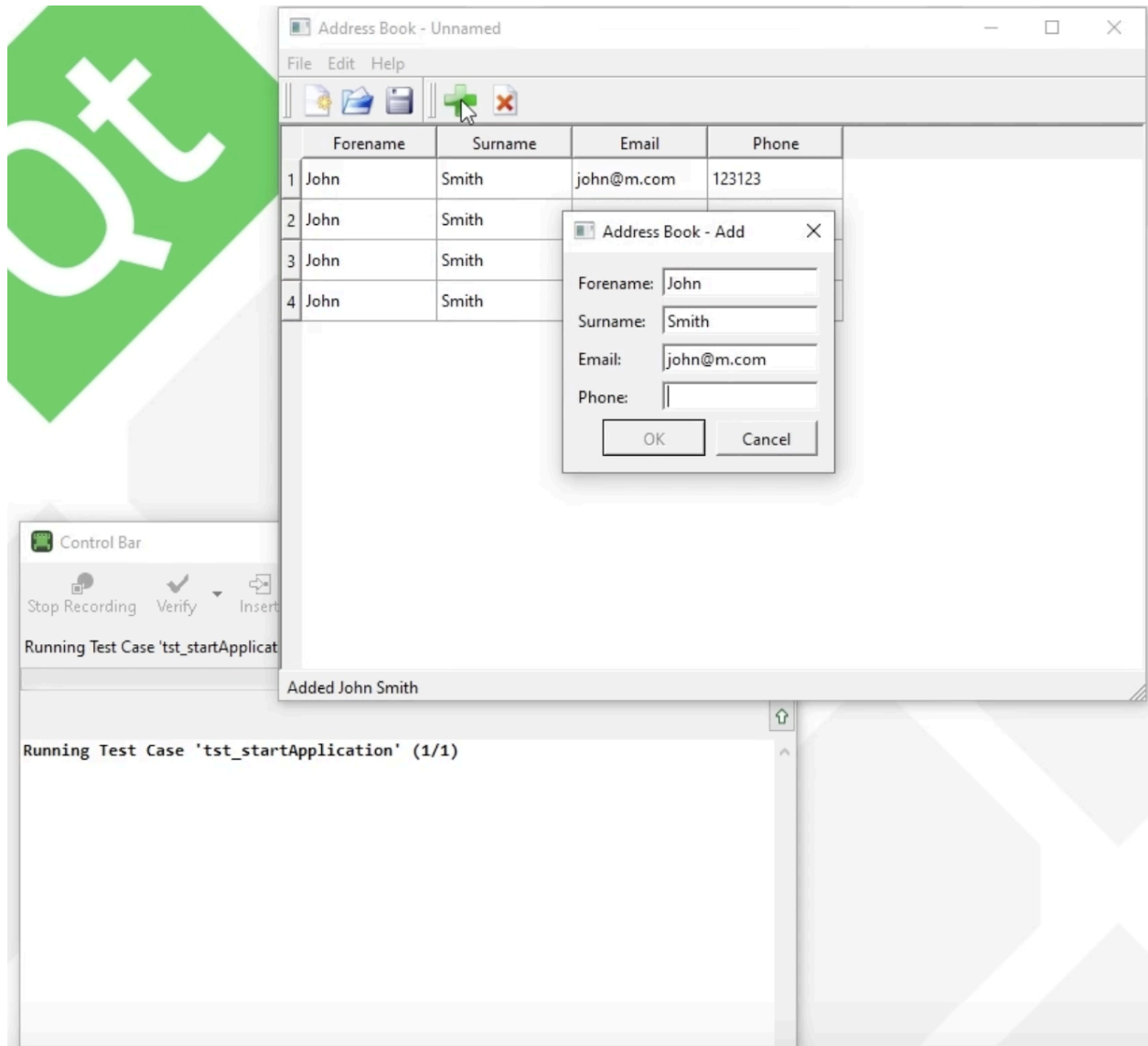


We will start by creating a new test case named `tst_attachable_application` in our test suite.

For this example, we did not record anything but directly wrote the test script in the Python code editor.

We had already created a test called `tst_startApplication` that started the address book, created a new book, and added five entries to it.

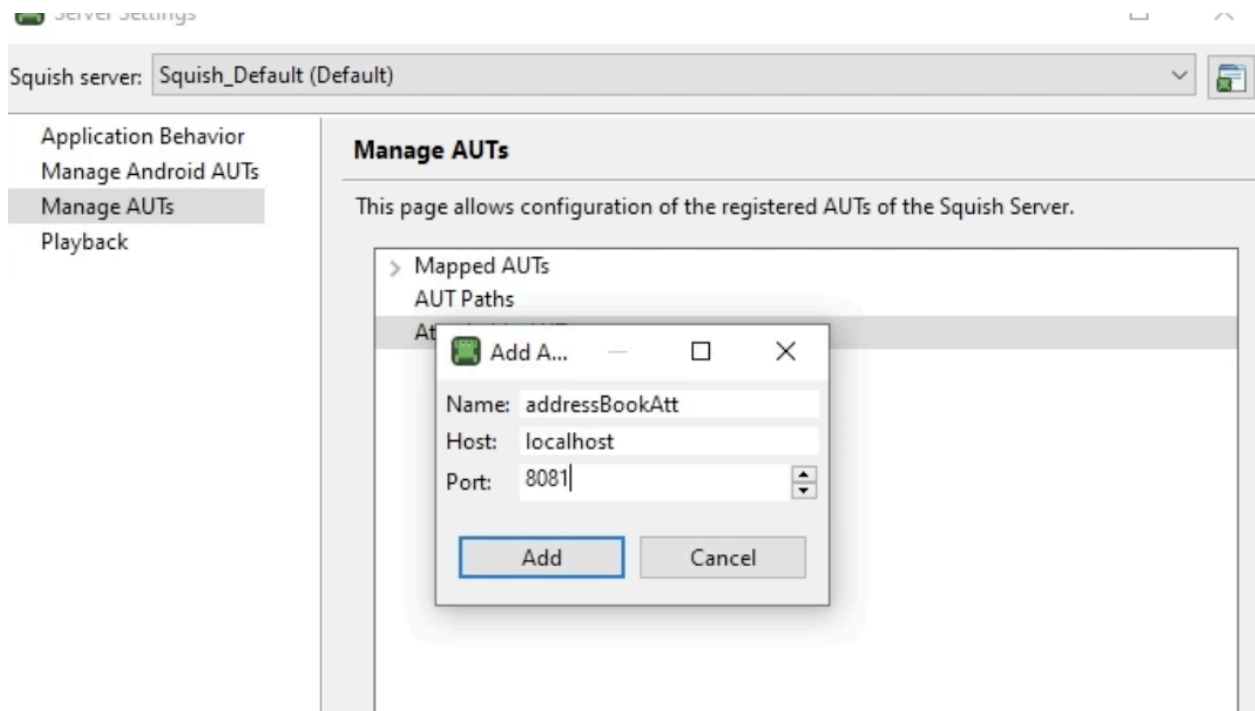
STEP 2- Running the test



Next, we ran the `tst_startApplication` test. The test was working correctly, as we saw the application appearing on the screen and the creation of entries.

In this test, our application started at the beginning of our test and was closed at the end. In this demo, we wanted to replicate this test but with an application started prior to running the test script. The application would continue to run after the test has ended.

STEP3- Registering the `addressBookAtt`



We started by registering the "addressBookAtt" as an attachable application. As for a mapped AUT, we went to Edit > Server Settings... > Manage AUTs. Then, in the Attachable AUTs section, we added a new AUT. We named it "addressBookAtt", kept the localhost for the host, and set the port to 8081. The host is the address of the machine where the AUT is started, and the port is the port number used when starting the AUT.

We saved the settings and went back to the test script.

STEP4- Writing the test case, part 1

```

4
5 import names
6
7 def main():
8     app = attachToApplication("addressBookAtt")
9     test.log("message detail", "%s has been attached with id %d on port %d with host %s" % (app.name, app.pid, app.port, app.host))
10     clickButton(waitForObject(names.address_Book_New_QToolButton))
11
12     for i in range(5):
13         clickButton(waitForObject(names.address_Book_Add_QToolButton))
14         type(waitForObject(names.forename_LineEdit), "John")
15         type(waitForObject(names.forename_LineEdit), "<Tab>")
16         type(waitForObject(names.surname_LineEdit), "Smith")
17         type(waitForObject(names.surname_LineEdit), "<Tab>")
18         type(waitForObject(names.email_LineEdit), "john@m.com")
19         type(waitForObject(names.email_LineEdit), "<Tab>")
20         type(waitForObject(names.phone_LineEdit), "123123")
21         type(waitForObject(names.phone_LineEdit), "<Return>")

```

ext, we wrote the test case in **tst_attachable_application**.

First, we copied and pasted the code from **tst_startApplication** to **tst_attachable_application**. We removed the first line since we did not want to start the application in our test. Instead, we wanted to attach to it.

So, we used the **attachToApplication()** function. The function makes Squish attach to the application, meaning that Squish can interact with and test the attached application. It also returns the application context. We saved the application context object returned by the **attachToApplication** function.

```
app = attachToApplication("addressBookAtt")
```

Since we had the application context, we could use this to query information about the attached AUT in the same manner as it was done in the first section. So, we added a call to test log:

```
test.log("%s has been attached with id %d on port %d with host %s" % (app.name, app.pid, app.port, app.host))
```

STEP 5- Writing the test case, part 2

```
3 import names
4
5 def main():
6     app = attachToApplication("addressBookAtt")
7
8     test.log("%s has been attached with id %d on port %d with host %s" % (app.name, app.pid, app.port, app.host))
9
10    if (waitForObjectExists(names.address_Book_Add_QToolButton).enabled == False):
11        clickButton(waitForObject(names.address_Book_New_QToolButton))
12
13    for i in range(5):
14        clickButton(waitForObject(names.address_Book_Add_QToolButton))
15        type(waitForObject(names.forename_LineEdit), "John")
16        type(waitForObject(names.forename_LineEdit), "<Tab>")
17        type(waitForObject(names.surname_LineEdit), "Smith")
18        type(waitForObject(names.surname_LineEdit), "<Tab>")
19        type(waitForObject(names.email_LineEdit), "john@m.com")
20        type(waitForObject(names.email_LineEdit), "<Tab>")
21        type(waitForObject(names.phone_LineEdit), "123123")
22        type(waitForObject(names.phone_LineEdit), "<Return>")
```

Next, we wanted to create a new address book if one was not already open. So, we added an "if" statement which checked if an address book was open, and if it was not, we would click the new address book button:

```
if (waitForObjectExists(names.address_Book_Add_QToolButton).enabled == False):
```

```
clickButton(waitForObject(names.address_Book_New_QToolButton))
```

STEP 6- Writing the test case, part 3

```
6 app = attachToApplication("addressBookAtt")
7
8 test.log("%s has been attached with id %d on port %d with host %s" % (app.name, app.pid, app.port, app.host))
9
10 if (waitForObjectExists(names.address_Book_Add_QToolButton).enabled == False):
11     clickButton(waitForObject(names.address_Book_New_QToolButton))
12
13 for i in range(5):
14     clickButton(waitForObject(names.address_Book_Add_QToolButton))
15     type(waitForObject(names.forename_LineEdit), "John")
16     type(waitForObject(names.forename_LineEdit), "<Tab>")
17     type(waitForObject(names.surname_LineEdit), "Smith")
18     type(waitForObject(names.surname_LineEdit), "<Tab>")
19     type(waitForObject(names.email_LineEdit), "john@m.com")
20     type(waitForObject(names.email_LineEdit), "<Tab>")
21     type(waitForObject(names.phone_LineEdit), "123123")
22     type(waitForObject(names.phone_LineEdit), "<Return>")
23     pid = app.pid
24     app.detach()
25     test.log("Squish detached from app with id %d" % (pid))
```

Next, we observed the “for” loop. In the loop, we wanted to attach, detach, and reattach to the same application. The process would be to attach to the first application and interact with it. Then, before attaching to the next application to be tested, we would detach from the first application with the **detach()** function.

For demonstration, we added a call to the **detach()** function at the end of our “for” loop:

```
app.detach()
```

Before detaching, we saved the process identifier, or pid, of the application to prompt it after the application was detached. We needed to save the pid into a variable since after calling the **detach()** function, the application context would not be available anymore:

```
pid = app.pid
```

```
test.log("Squish detached from app with id %d" % (pid))
```

STEP 7- Writing the test case, part 4

```
6 app = attachToApplication("addressBookAtt")
7
8 test.log("%s has been attached with id %d on port %d with host %s" % (app.name, app.pid, app.port, app.host))
9
10 if (waitForObjectExists(names.address_Book_Add_QToolButton).enabled == False):
11     clickButton(waitForObject(names.address_Book_New_QToolButton))
12
13 for i in range(5):
14     if not app.isRunning:
15         app = attachToApplication("addressBookAtt")
16         test.log("%s has been attached with id %d on port %d with host %s" % (app.name, app.pid, app.port, app.host))
17     clickButton(waitForObject(names.address_Book_Add_QToolButton))
18     type(waitForObject(names.forename_LineEdit), "John")
19     type(waitForObject(names.forename_LineEdit), "<Tab>")
20     type(waitForObject(names.surname_LineEdit), "Smith")
21     type(waitForObject(names.surname_LineEdit), "<Tab>")
22     type(waitForObject(names.email_LineEdit), "john@gmail.com")
23     type(waitForObject(names.email_LineEdit), "<Tab>")
24     type(waitForObject(names.phone_LineEdit), "123123")
25     type(waitForObject(names.phone_LineEdit), "<Return>")
26     pid = app.pid
27     app.detach()
28     test.log("Squish detached from app with id %d" % (pid))
```

Then, since we detached at the end of a loop, we needed to reattach when we were entering back into the loop.

We first confirmed that Squish had not already attached to the AUT by using the **isRunning** property. Indeed, **isRunning** is true if the communication between squishserver and Squish Hook in the AUT is active; which is the case when Squish has attached to the AUT. So, we will write:

if not app.isRunning:

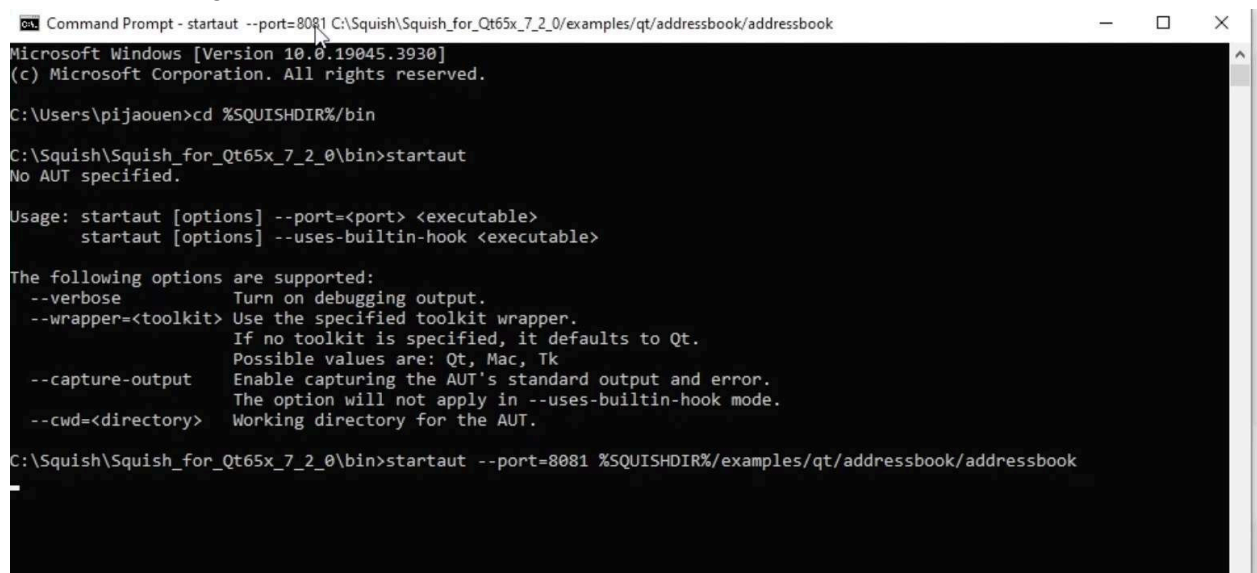
In this "if" statement, we called the same **attachToApplication()** function to reattach to the application like we did at the beginning of our test

app = attachToApplication("addressBookAtt")

test.log("%s has been attached with id %d on port %d with host %s" % (app.name, app.pid, app.port, app.host))

Our test was now ready, and we saved it.

STEP 8- Starting the AUT



```
Command Prompt - startaut --port=8081 C:\Squish\Squish_for_Qt65x_7_2_0/examples/qt/addressbook/addressbook
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pijaouen>cd %SQUISHDIR%\bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>startaut
No AUT specified.

Usage: startaut [options] --port=<port> <executable>
       startaut [options] --uses-builtin-hook <executable>

The following options are supported:
--verbose           Turn on debugging output.
--wrapper=<toolkit> Use the specified toolkit wrapper.
                   If no toolkit is specified, it defaults to Qt.
                   Possible values are: Qt, Mac, Tk
--capture-output    Enable capturing the AUT's standard output and error.
                   The option will not apply in --uses-builtin-hook mode.
--cwd=<directory>   Working directory for the AUT.

C:\Squish\Squish_for_Qt65x_7_2_0\bin>startaut --port=8081 %SQUISHDIR%\examples/qt/addressbook/addressbook
```

We started the AUT using the **startaut** command line tool. We opened a command window and set the bin folder in the Squish directory as the current directory. Here, we called **startaut** with two options. The first one was the port corresponding to the port used when registering the attachable AUT. The second was the path to the executable to be used. Here, we used the address book application. So, we had the following command:

SQUISHDIR\bin>startaut --port={port} {path to aut}

STEP 9- Running the test

Recent Test Results ▾	
Result	Message
▼ TestSuite	suite_advanced_hooking_section3
▼ TestCase	tst_attachable_application
Log	addressbook has been attached with id 15412 on port 58983 with host 127.0.0.1
Log	Squish detached from app with id 15412
Log	addressbook has been attached with id 15412 on port 58983 with host 127.0.0.1
Log	Squish detached from app with id 15412
Log	addressbook has been attached with id 15412 on port 58983 with host 127.0.0.1
Log	Squish detached from app with id 15412
Log	addressbook has been attached with id 15412 on port 58983 with host 127.0.0.1
Log	Squish detached from app with id 15412
Log	addressbook has been attached with id 15412 on port 58983 with host 127.0.0.1
Log	Squish detached from app with id 15412
	Squish detached from app with id 15412

We moved back to the Squish IDE and launched our test. We saw Squish attached successfully to the application and could interact with it. At the end of the test, the application was not closed. We could have run our test again once more and seen the same behavior.

Summary

ApplicationContext is created whenever an AUT is started

In general, an AUT (Application Under Test) is started in each test case with a call to the function **startApplication()**, with the name of the AUT to start passed as argument. The function returns a handle to the AUT's application context.

The **ApplicationContext** object is created whenever an AUT is started. This object allows Squish to query the AUT for the command line it was launched with, the used memory, what the AUT writes to stdout and stderr among other useful information.

After an AUT has been started with **startApplication()**, the function **detach()** can be used to tell Squish to close it. The function can be used when an application needs to be closed during the test execution. After detaching from an AUT, its objects will no longer be accessible.

It's possible to start several instances of the same application or start multiple different applications with one script

The **startApplication()** function can be called several times in the same test script, so it is possible to start several instances of the same application or start multiple different applications with one script. Being able to start several AUTs that can all be evaluated in the same test script is essential for testing client/server systems, for instance.

There's a possibility to test two applications that have been built with different toolkits in the same test script

With the package Squish for Qt, it is also possible to start AUTs that are built with different GUI toolkits (Tk or iOS) from the same test script. In that case, the wrappers corresponding to the toolkit the AUT was built with must be set before starting the AUT. This is done using the **setWrappersForApplication()** function.

Furthermore, when testing AUTs built with different toolkits that are not supported by the same Squish packages (for instance testing a Qt and a Java application in the test script), the required Squish packages must all

be installed, and the respective Squish tools must be used. The multiple Squish servers must then be configured and started before the execution of the script.

Squish can attach to an application opened outside of the test case

Attaching to an application opened outside of the test case is done with the function **attachToApplication()**. The AUT must be started with the startaut command line tool to be attachable. The AUT must be registered as an attachable AUT to call the attachToApplication() function successfully.

When working with an attached application, the **detach()** function will detach from the application but leave it running. The application can then be attached to with **attachToApplication()** again.